

Flow-Level Intrusion Detection에서 Neural Model과 Traditional Model의 Attack-Type-Specific Evaluation

Yonghyeon Lee

yhylee@ucdavis.edu

University of California, Davis

Davis, California, USA

Abstract

Network intrusion detection 연구에서는 종종 aggregate model performance가 보고되지만, 실제 보안 운영자는 어떤 공격이 여전히 탐지되지 않는지를 알아야 한다. 이 논문은 flow-level intrusion detection에서 neural model이 traditional machine learning baseline과 비교해 false-negative profile을 어떻게 바꾸는지 평가한다. CSE-CIC-IDS2018 processed flow-feature dataset을 사용해 Logistic Regression, Random Forest, Shallow MLP, Lightweight PyTorch MLP, Deep MLP, Autoencoder + MLP를 binary benign/malicious task로 학습했고, 원래 attack label은 post-hoc analysis를 위해 보존했다. 평가는 precision, recall, F1-score, false-negative rate, confusion matrix, fine-grained attack-type detection rate, coarse attack-family detection rate를 비교한다. 핵심 결과는 neural model이 일관되게 더 우수하지 않았다는 것이다. Deep MLP와 Autoencoder + MLP는 precision과 F1-score를 개선했지만, Logistic Regression이 가장 높은 attack recall과 가장 낮은 false-negative rate를 보였다. 공격 유형별로는 DoS/DDoS와 많은 brute-force attack이 잘 탐지되었지만, Infiltration은 모든 모델에서 중요한 blind spot으로 남았다.

Keywords

intrusion detection, neural networks, Random Forest, false negatives, CSE-CIC-IDS2018, attack-type evaluation

1 Introduction

Network intrusion detection system은 network activity를 benign 또는 suspicious로 분류해 방어자가 malicious traffic을 조사하거나 차단할 수 있게 한다. 많은 machine learning 기반 IDS 연구에서는 accuracy나 F1-score 같은 aggregate score가 headline result로 제시된다. 이런 방식은 모델을 전체적으로 비교하는 데에는 유용하지만, security analysis에는 충분하지 않다. 운영 환경의 IDS에서는 오류의 비용이 서로 다르다. False positive는 analyst workload를 증가시키지만, false negative는 malicious traffic이 탐지되지 않고 통과했다는 뜻이다. 따라서 어떤 모델이 높은 aggregate score를 보여도 특정 attack family를 반복적으로 놓친다면 보안적으로 위험할 수 있다.

이 프로젝트는 flow-level intrusion detection 환경에서 이 문제를 분석한다. Flow-level dataset은 packet count, byte count, flow duration, rate, inter-arrival timing, TCP flag behavior 같은 feature로 network traffic을 요약한다. 이 feature들은 raw packet trace보다 quarter-scale reproducible study에 적합하며, traditional machine learning model과 neural network model 모두에 사용할 수 있다. 하지만 flow-level data에도 위험이 있다. Binary classification 결과만 보면 attack category별 성능 차이가 숨겨질 수 있기 때문이다.

본 연구의 research question은 다음과 같다. *Flow-level network traffic data*에서 *neural model*은 *traditional machine learning baseline*과 비교해 어떤 *network intrusion type*을 더 잘 탐지하는가? 더 구체적으로는 DoS/DDoS, brute force, botnet, infiltration, web-based attack 같은 attack family 별로 model choice가 detection performance를 어떻게 바꾸는지 확인한다. 이 질문은 단순히 neural model이 overall로 이기는지를 묻는 것이 아니다. 목표는 각 모델의 security-relevant failure pattern을 이해하는 것이다.

초기 hypothesis는 neural model이 flow-level feature 사이의 nonlinear combination에 의존하는 attack category에서 탐지를 개선할 것이라는 것이었다. 예를 들어 botnet이나 web-based attack은 하나의 feature threshold만으로는 잘 포착되지 않을 수 있다. 하지만 high-volume DDoS traffic처럼 feature pattern이 비교적 명확한 공격에서는 Random Forest 같은 traditional model도 여전히 경쟁력이 있을 것으로 예상했다. 그 이유는 Random Forest가 tabular feature에 대한 threshold-based decision rule을 잘 학습하는 반면, multilayer perceptron은 continuous feature interaction을 학습할 수 있지만 class imbalance와 dataset artifact에 더 민감할 수 있기 때문이다.

이 연구의 theoretical contribution은 flow-level IDS model behavior를 해석하기 위한 attack-type-specific evaluation framework이다. 모델 성능을 하나의 숫자로 처리하는 대신, 이 연구는 security failure mode를 비교한다. Empirical contribution은 같은 CSE-CIC-IDS2018 sample에서 여섯 개 모델을 학습하고 model-level 및 attack-level false negative를 보고하는 reproducible implementation이다. 최종 결과는 neural model이 일부 aggregate metric을 개선할 수는 있지만, missed attack을 일관되게 줄이지는 못한다는 점을 보여준다.

2 Related Work

Intrusion detection dataset에 대한 prior work는 이 프로젝트의 empirical setting을 뒷받침한다. CICIDS2017은 older IDS dataset보다 더 realistic한 benchmark를 제공하기 위해 소개되었다. 이 dataset은 benign traffic과 common attack traffic을 labeled network-flow feature로 포함한다 [1]. CSE-CIC-IDS2018은 이 방향을 확장해 brute force, Heartbleed, botnet, DoS, DDoS, web attack, infiltration을 포함하는 더 큰 cyber-defense dataset을 제공한다 [2]. 이 dataset들은 binary benign/malicious detection과 attack-category-specific analysis를 모두 지원하기 때문에 본 연구와 직접적으로 관련된다.

IDS modeling에 대한 prior work는 baseline 선택을 동기화한다. Random Forest는 many decision-tree predictor를 ensemble로 결합하기 때문에 standard traditional machine learning baseline으로 사용된다 [3]. Deep learning도 network intrusion detection에서 활발히 연구되었다. Gamage and Samarabandu는 IDS를 위

한 deep learning approach를 survey하고 feed-forward neural network, autoencoder, deep belief network, LSTM을 legacy 및 modern IDS dataset에서 비교했다 [4]. 이는 lightweight multilayer perceptron 사용을 뒷받침하면서도, model comparison 자체가 새로운 것은 아님을 보여준다.

이 프로젝트가 다루는 gap은 IDS model performance를 해석하는 방식에 있다. 많은 IDS 연구는 overall accuracy나 aggregate detection performance를 강조하지만 [4], dataset이 imbalanced하거나 일부 attack category가 더 어려운 경우 aggregate metric은 security-relevant failure를 숨길 수 있다. Saito and Rehmsmeier는 imbalanced classification problem에서 precision-recall analysis가 ROC plot보다 더 informative하다는 점을 보였다 [5]. 이 점은 IDS에서 중요하다. False negative는 system이 malicious traffic을 flag하지 못했다는 의미이기 때문이다.

따라서 이 프로젝트는 existing work 안에 위치하지만 더 좁고 security-centered question을 묻는다. 나는 PyTorch가 traditional machine learning보다 단순히 “IDS를 더 잘한다”고 주장하지 않는다. 대신 Logistic Regression, Random Forest, lightweight PyTorch MLP를 비교해 각 모델이 어떤 attack category를 놓치는지 식별한다. Related work는 세 가지 방식으로 이 프로젝트를 지지한다. CICIDS dataset은 labeled flow-level setting을 제공하고, prior ML/DL IDS work는 model comparison을 정당화하며, imbalanced-evaluation research는 recall과 false-negative rate를 주요 security metric으로 사용하는 이유를 제공한다.

3 Method

3.1 Dataset and Filtering

이 연구는 CSE-CIC-IDS2018 processed traffic data for machine learning을 사용한다. Raw file은 public AWS S3 bucket에서 다운로드했고, reproducible sampled dataset으로 처리했다. 최종 sample은 50,000개 flow, 79개 numeric flow feature, 37,308개 benign flow, 12,692개 attack flow를 포함한다. 전처리 과정에서는 binary model output을 이후 attack type별로 분석할 수 있도록 14개 fine-grained attack label을 보존했다.

모델 학습 task는 binary classification이다. 각 row는 benign 또는 malicious로 labeling된다. Original attack label은 prediction target으로 사용하지 않는다. 연구 질문이 binary IDS model이 어떤 attack type을 놓치는지에 있기 때문이다. 대신 original label은 train, validation, test split 전체에서 보존되고 post-hoc evaluation에만 사용된다.

Preprocessing pipeline은 column name cleaning, label column detection, non-feature metadata column removal, missing/infinite value replacement, numeric feature scaling, train/validation/test split을 수행한다. 최종 split은 training 32,000 rows, validation 8,000 rows, test 10,000 rows이다. Random seed는 reproducibility를 위해 42로 고정했다. Sampling 과정에서는 rare attack category가 제거되지 않도록 rare-label preservation을 사용했다.

3.2 System Building Blocks

구현된 system은 lightweight flow-level IDS prototype이다. 네 개의 주요 building block으로 구성된다. 첫째, dataset preparation script는 raw CSE-CIC-IDS2018 CSV file을 읽고 attack label을 보존하는 sampling을 수행한 뒤 processed training CSV를 생성한다. 둘째, training pipeline은 processed CSV를 로드하고 shared preprocessing pipeline을 적용한 뒤, 선택된 모든 모델을 학습하고 artifact를 저장한다. 셋째, evaluation module은 binary metric,

confusion matrix, attack-type detection summary, attack-family detection summary, plot을 생성한다. 넷째, prediction module은 saved artifact를 로드해 새로운 CSV file에 trained model을 적용한다.

System은 Python, pandas, NumPy, scikit-learn, PyTorch, matplotlib로 구현되었다. scikit-learn은 Logistic Regression, Random Forest, scaling, splitting, metric computation에 사용되었다. PyTorch는 neural model과 training loop에 사용되었다. Repository에는 data preparation, training, prediction을 재현하기 위한 README command도 포함되어 있다.

3.3 Models

총 여섯 개 모델을 비교했다. Logistic Regression은 simple linear baseline이며 balanced class weight를 사용한다. Random Forest는 traditional non-neural baseline이며 balanced subsampling을 사용한다. Neural baseline은 Shallow MLP, Lightweight PyTorch MLP, Deep MLP, Autoencoder + MLP이다. Shallow MLP는 하나의 hidden layer를 가진다. Lightweight PyTorch MLP는 기존 two-hidden-layer model이다. Deep MLP는 feed-forward layer를 더 많이 사용해 depth 증가가 attack recall을 개선하는지 확인한다. Autoencoder + MLP는 먼저 compressed latent representation을 학습한 뒤 그 representation을 classifier에 입력한다.

모든 모델은 같은 split에서 학습 및 평가되었다. 이를 통해 비교의 초점을 data difference가 아니라 model behavior에 맞췄다. Neural model은 leaderboard-style optimization이 아니라 attack-specific failure pattern 비교를 위한 reproducible baseline으로 구현했기 때문에 과도한 tuning은 하지 않았다.

3.4 Measures

주요 metric은 precision, recall, F1-score, false-negative rate, confusion matrix count, attack-type detection rate, attack-family detection rate이다. Binary malicious class에서 recall은 전체 attack 중 탐지된 비율을 의미한다. False-negative rate는 1 minus recall이며, 놓친 attack의 비율을 직접 측정한다. Precision은 malicious로 예측한 flow 중 실제 malicious flow의 비율이다. F1-score는 precision과 recall을 균형 있게 반영한다.

Attack-type detection rate는 malicious test row를 original fine-grained attack label별로 grouping한 뒤, 각 모델이 malicious로 탐지한 row의 비율을 계산한다. Attack-family detection rate는 fine-grained label을 Botnet, Brute Force, DoS/DDoS, Infiltration, Web Attack의 다섯 broad group으로 mapping한다. 이 두 수준의 분석이 모두 필요하다. Fine-grained view는 구체적인 failure case를 보여주고, family-level view는 broader security pattern을 요약한다.

3.5 Implementation and Reproducibility

실험은 manually edited notebook이 아니라 reproducible command-line pipeline으로 실행되었다. 여러 모델과 여러 attack category를 비교하기 때문에 manual step은 서로 다른 split을 평가하는 실수를 만들 수 있다. Repository에는 dataset preparation, training, evaluation, prediction을 위한 별도 script가 있다. Dataset preparation command는 raw CSE-CIC-IDS2018 CSV file을 읽고 rare-label-preserving sampling을 적용해 processed CSV를 생성한다. Training command는 이 processed file을 읽어 fixed train/validation/test split을 만들고, 모든 모델을 학습하며, result table과 plot을 저장한다.

Table 1: Final run에서 생성된 주요 implementation output.

Output	Purpose
metrics_summary.csv	Overall model metrics
attack_type_detection.csv	Fine-grained attack recall
attack_family_detection.csv	Coarse family recall
confusion_matrix_*.png	Model error visualization
false_negative_rate.png	Security-focused comparison
*.pt, *.pkl artifacts	Saved trained models

Table 2: Test split에서의 malicious-class overall performance.

Model	Precision	Recall	F1	FNR
Logistic Regression	0.512	0.528	0.520	0.472
Random Forest	0.649	0.471	0.546	0.529
Shallow MLP	0.578	0.524	0.550	0.476
PyTorch MLP	0.615	0.507	0.556	0.493
Deep MLP	0.747	0.467	0.574	0.533
Autoencoder + MLP	0.723	0.471	0.570	0.529

최종 실행은 public AWS bucket에서 다운로드한 CSE-CIC-IDS2018 processed machine-learning CSV file을 사용했다. Local raw dataset은 10개 CSV file, 약 6.41GB였다. Processed experiment sample은 data/processed/ids_sample.csv로 저장되었다. 최종 run은 Logistic Regression, Random Forest, Shallow MLP, PyTorch MLP, Deep MLP, Autoencoder + MLP artifact를 생성했다. 또한 metrics_summary.csv, attack_type_detection.csv, attack_family_detection.csv, confusion matrix, false-negative plot, prediction output을 생성했다.

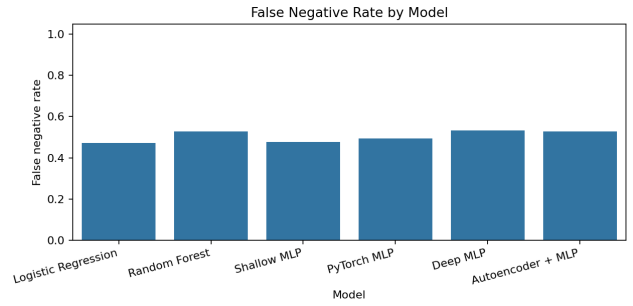
Table 1는 final run의 주요 output을 요약한다. 이 output들은 프로젝트가 계획된 method에 따라 실행되었음을 보여준다. 같은 dataset split이 모든 모델에 사용되었고, attack label은 evaluation을 위해 보존되었으며, aggregate result와 attack-specific result가 모두 수집되었다. Prediction script는 saved model을 로드해 CSV file에 적용할 수 있으므로 offline reporting을 넘어 system-building component도 보여준다.

4 Results

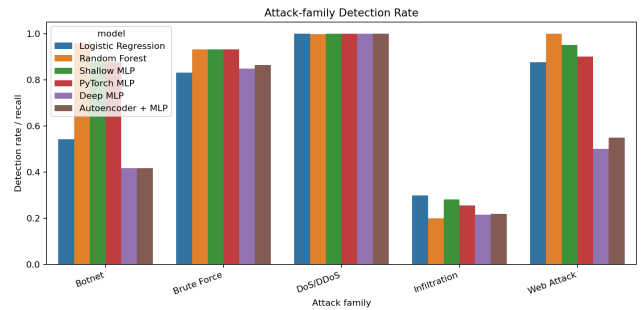
4.1 Overall Model Performance

Table 2은 10,000-row test split에서 binary malicious-class performance를 보여준다. Deep MLP는 가장 높은 F1-score와 precision을 달성했다. 하지만 Logistic Regression은 가장 높은 recall과 가장 낮은 false-negative rate를 보였다. 이 차이는 실험의 security interpretation에서 핵심적이다. F1-score 기준으로 가장 좋아 보이는 모델이 공격을 가장 적게 놓치는 모델은 아니었다.

False-negative count는 이 차이를 더 명확하게 보여준다. Logistic Regression은 1,197개 attack을 놓쳤다. Shallow MLP는 1,209개, PyTorch MLP는 1,252개, Random Forest는 1,342개, Autoencoder + MLP는 1,343개, Deep MLP는 1,354개를 놓쳤다. Deep MLP는 더 selective하게 동작했다. Malicious로 예측한 경우에는 더 자주 맞았지만, 더 많은 실제 attack을 탐지하지 못했다. IDS에서 missed attack을 최소화하는 것이 우선이라면 이 tradeoff는 문제가 될 수 있다.

**Figure 1: 모델별 false-negative rate. IDS security coverage에서는 낮을수록 좋다.****Table 3: Coarse attack-family recall 기준 best model.**

Family	Best model	Recall
Botnet	Random Forest	0.958
Brute Force	RF / Shallow MLP / PyTorch MLP	0.932
DoS/DDoS	Shallow MLP / PyTorch MLP	1.000
Infiltration	Logistic Regression	0.299
Web Attack	Random Forest	1.000

**Figure 2: Coarse attack family와 model별 detection recall.**

4.2 Coarse Attack-Family Results

Table 3는 coarse attack family별 가장 강한 모델을 요약한다. Family-level result는 IDS task의 난이도가 attack type에 강하게 의존한다는 점을 보여준다. DoS/DDoS는 tested model들에서 거의 해결된 category였다. Shallow MLP와 PyTorch MLP는 DoS/DDoS family에서 1.000 recall을 달성했다. Random Forest는 Botnet과 Web Attack에서 가장 강했다. Brute Force에서는 Random Forest, Shallow MLP, PyTorch MLP가 0.932 recall로 동률이었다.

가장 중요한 결과는 Infiltration family이다. Infiltration은 1,669개 test attack을 포함했지만, 가장 좋은 Logistic Regression도 499개만 탐지했다. Family-level recall은 0.299에 불과했다. Random Forest는 334개, Shallow MLP는 469개, PyTorch MLP는 428개, Deep MLP는 359개, Autoencoder + MLP는 367개를 탐지했다. 이 family는 전체 false negative의 큰 부분을 차지한다.

4.3 Fine-Grained Attack-Type Results

Fine-grained result는 family-level pattern을 만드는 구체적인 attack을 설명한다. 몇몇 attack label은 거의 모든 모델이 완벽하게 탐지했다. DDOS attack-HOIC, DDOS attack-LOIC-UDP, DoS attacks-SlowHTTPTest, FTP-BruteForce, SSH-Bruteforce는 대부분 모델에서 near-perfect 또는 perfect recall을 보였다. 이 attack들은 benign traffic과 구분되는 flow-feature signature가 비교적 뚜렷한 것으로 보인다.

Table 4는 이 분석에 사용된 per-label recall을 보여준다. 이 표는 best model이 attack type에 따라 달라진다는 점을 보여준다. 또한 dataset 전체의 난이도가 균등하지 않다는 점도 보여준다. 일부 row는 거의 해결되었지만, Infiltration은 여섯 모델 모두에서 일관되게 약했다.

DDoS attacks-LOIC-HTTP는 neural model이 조금 더 안정적으로 보인 case이다. Shallow MLP, PyTorch MLP, Deep MLP, Autoencoder + MLP는 모두 1.000 recall을 달성했고, Logistic Regression과 Random Forest는 각각 1개 sample을 놓쳤다. 이는 제한적인 의미에서 hypothesis를 지지한다. 일부 attack type에서는 neural model이 traditional baseline을 match하거나 약간 개선했다.

Bot traffic은 반대 pattern을 보였다. Random Forest는 0.958 recall을 달성했지만, Logistic Regression은 0.542였고 deep neural model은 낮았다. Deep MLP와 Autoencoder + MLP는 각각 Bot에서 0.417 recall에 그쳤다. 이는 이 small category에서 tree-based model이 유용한 feature threshold를 더 잘 포착했을 가능성을 보여준다.

Brute Force -Web은 feature interaction을 표현할 수 있는 모델에 유리했다. Random Forest, Shallow MLP, PyTorch MLP는 모두 0.778 recall을 달성했지만, Logistic Regression은 0.444에 그쳤다. 하지만 Brute Force -XSS는 deeper neural model이 rare label에서 항상 더 robust하지 않다는 점을 보여준다. Logistic Regression, Random Forest, Shallow MLP, PyTorch MLP는 1.000 recall을 달성했지만, Deep MLP와 Autoencoder + MLP는 0.450에 그쳤다.

SQL Injection은 Random Forest가 1.000 recall로 가장 강했다. Shallow MLP도 0.900으로 양호했지만, Logistic Regression은 0.750, PyTorch MLP는 0.800, Deep MLP는 0.550, Autoencoder + MLP는 0.650이었다. 이는 web-related attack에서 Random Forest가 여전히 경쟁력 있고 때로는 우수하다는 broader result를 지지한다.

가장 중요한 fine-grained attack label은 Infiltration이다. Test set에서 가장 큰 attack label이며 1,669개 sample을 포함한다. 모든 모델이 이 label에서 낮은 성능을 보였다. Logistic Regression이 0.299 recall로 가장 좋았고, Shallow MLP는 0.281, PyTorch MLP는 0.256, Autoencoder + MLP는 0.220, Deep MLP는 0.215, Random Forest는 0.200이었다. 이것이 실험에서 가장 명확한 security blind spot이다.

5 Discussion

결과는 hypothesis를 부분적으로만 지지한다. Neural model은 일부 aggregate measure를 개선했고, Shallow MLP와 PyTorch MLP는 Brute Force family에서 Random Forest와 같은 수준을 보였다. MLP model들은 DoS/DDoS traffic에서도 매우 좋은 성능을 냈다. 하지만 neural model이 overall false negative를 줄이지는 못했다. 가장 높은 recall을 보인 모델은 Logistic Regression이었고, 가장 낮은 recall group에는 deeper neural variant가 포함되었다.

이 결과가 중요한 이유는 metric choice가 결론을 바꾸기 때문이다. F1-score만 보고하면 Deep MLP가 가장 강한 모델처럼 보인다. Precision만 보고해도 Deep MLP는 더 좋아 보인다. 하지만

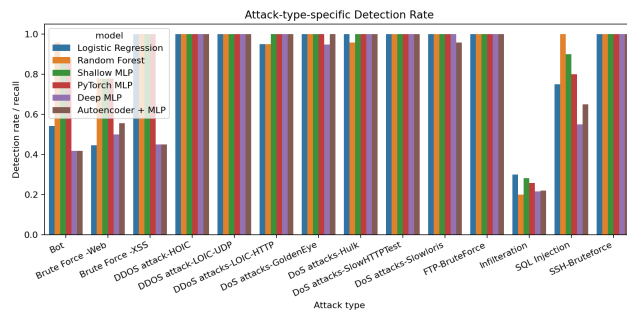


Figure 3: Fine-grained attack type별 model recall.

IDS의 security objective는 malicious prediction을 정확하게 하는 것만이 아니다. Malicious traffic이 탐지되지 않고 지나가는 것을 줄이는 것도 중요하다. 이 기준에서는 이번 실험에서 Logistic Regression과 Shallow MLP가 Deep MLP보다 더 매력적이다.

Attack-type breakdown은 aggregate metric이 왜 misleading할 수 있는지도 보여준다. DoS/DDoS category는 거의 완벽하게 탐지되며, clear pattern 때문에 overall system이 강해 보이게 만들 수 있다. 동시에 Infiltration은 frequent하면서도 어렵다. 모든 모델이 Infiltration sample 대부분을 놓쳤다. 따라서 aggregate F1만 기반으로 deployment decision을 내리면 IDS가 가장 개선해야 할 category를 숨길 수 있다.

Autoencoder + MLP 결과도 의미가 있다. Autoencoder는 high-dimensional flow feature의 compact representation을 학습할 수 있기 때문에 deep IDS literature에서 자주 사용된다. 이번 실험에서 autoencoder-based model은 Deep MLP와 유사하게 동작했다. Precision과 F1은 높았지만 recall은 낮았다. 이는 autoencoder가 IDS에 효과가 없다는 일반적 결론을 의미하지는 않는다. 다만 이 scoped representation-learning baseline은 false-negative problem을 해결하지 못했다.

6 Risks and Limitations

첫 번째 limitation은 dataset dependence이다. CSE-CIC-IDS2018은 public benchmark dataset이며, 모델이 general intrusion behavior가 아니라 dataset-specific artifact를 학습했을 수 있다. 이 위험을 줄이기 위해 본 연구는 accuracy-only reporting을 피하고 attack type별 false negative에 집중했다. 하지만 결과는 여전히 production IDS guarantee가 아니라 sampled benchmark에서 관찰된 behavior로 해석해야 한다.

두 번째 limitation은 class imbalance이다. 일부 attack type은 test support가 작기 때문에 perfect recall을 과도하게 해석하면 안 된다. 예를 들어 몇몇 rare attack category는 test sample이 30개 미만이다. 따라서 분석은 frequent하면서도 poorly detected된 Infiltration에 특별히 주목했다.

세 번째 limitation은 model tuning이다. Neural model은 exhaustively tuned architecture가 아니라 reproducible baseline으로 구현되었다. 다른 threshold, loss function, resampling strategy, hyperparameter가 recall을 개선할 수 있다. 특히 default 0.5 decision threshold는 security objective에 최적이지 아닐 수 있다.

네 번째 limitation은 binary training objective이다. 모델은 benign와 malicious flow를 구분하도록 학습되었고, attack label은 training 이후 analysis에만 사용되었다. Multiclass 또는 hierarchical model은 attack-specific pattern을 더 직접적으로 학습할

Table 4: Fine-grained attack-type recall by model. LR = Logistic Regression, RF = Random Forest, Shallow = Shallow MLP, PT = PyTorch MLP, Deep = Deep MLP, AE = Autoencoder + MLP.

Attack type	Test	LR	RF	Shallow	PT	Deep	AE
Bot	24	0.542	0.958	0.875	0.875	0.417	0.417
Brute Force -Web	18	0.444	0.778	0.778	0.778	0.500	0.556
Brute Force -XSS	20	1.000	1.000	1.000	1.000	0.450	0.450
DDoS attack-HOIC	618	1.000	1.000	1.000	1.000	1.000	1.000
DDoS attack-LOIC-UDP	15	1.000	1.000	1.000	1.000	1.000	1.000
DDoS attacks-LOIC-HTTP	20	0.950	0.950	1.000	1.000	1.000	1.000
DoS attacks-GoldenEye	19	1.000	1.000	1.000	1.000	0.947	1.000
DoS attacks-Hulk	23	1.000	0.957	1.000	1.000	1.000	1.000
DoS attacks-SlowHTTPTest	28	1.000	1.000	1.000	1.000	1.000	1.000
DoS attacks-Slowloris	23	1.000	1.000	1.000	1.000	1.000	0.957
FTP-BruteForce	18	1.000	1.000	1.000	1.000	1.000	1.000
Infiltration	1669	0.299	0.200	0.281	0.256	0.215	0.220
SQL Injection	20	0.750	1.000	0.900	0.800	0.550	0.650
SSH-Bruteforce	23	1.000	1.000	1.000	1.000	1.000	1.000

수 있다. Binary design은 lightweight IDS prototype에 맞고 비교를 단순하게 유지하기 위해 선택했다.

7 Future Work

첫 번째 next step은 security objective에 맞춘 threshold tuning이다. 고정된 0.5 threshold 대신 false positive를 통제하면서 false negative를 줄이는 threshold를 각 모델별로 찾아야 한다. 이것은 Deep MLP와 Autoencoder + MLP가 precision 일부를 포기하고 recall을 개선할 수 있는지 직접 테스트한다.

두 번째 next step은 Infiltration에 대한 targeted improvement이다. Infiltration이 missed attack의 대부분을 차지하므로, 이 category의 feature distribution을 inspect하고 benign flow와 비교하며 resampling 또는 cost-sensitive training이 detection을 개선하는지 확인해야 한다. 이 작업은 main security blind spot을 직접 다루기 때문에 단순히 모델 종류를 더 늘리는 것보다 우선순위가 높다.

세 번째 next step은 multiclass 또는 hierarchical classification이다. 모델이 먼저 malicious traffic을 탐지한 뒤 attack family를 분류할 수 있다. 또는 attack-family label을 training에 사용해 category-specific feature를 학습하도록 만들 수 있다.

네 번째 next step은 broader validation이다. 현재 연구는 CSE-CIC-IDS2018을 사용했다. 같은 pipeline을 CICIDS2017 또는 다른 flow-level IDS dataset에 적용하면 attack-specific conclusion이 하나의 benchmark를 넘어 generalize되는지 확인할 수 있다.

8 Conclusion

이 논문은 neural model이 traditional machine learning baseline과 비교해 flow-level intrusion detection에서 missed attack을 줄이는지 평가했다. 결론은 mixed이다. Deep MLP와 Autoencoder + MLP는 precision과 F1-score를 개선했지만, Logistic Regression이 가장 높은 attack recall과 가장 낮은 false-negative rate를 보였다. Shallow MLP는 neural model 중 recall 기준으로 가장 안정적이었고 Logistic Regression에 가까운 성능을 보였다.

가장 강한 결론은 IDS evaluation이 attack-type-specific이어야 한다는 점이다. DoS/DDoS와 여러 brute-force attack은 잘 탐지되었지만, Infiltration은 모든 모델에서 어려웠다. 따라서 aggregate metric 만으로는 충분하지 않다. F1-score가 좋아 보이는 모델도 가장 큰 security risk를 만드는 attack category를 놓칠 수 있다.

References

- [1] [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 108–116. <https://www.unb.ca/cic/datasets/ids-2017.html>
- [2] [2] Canadian Institute for Cybersecurity. 2018. CSE-CIC-IDS2018 on AWS: A Realistic Cyber Defense Dataset. University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2018.html>
- [3] [3] L. Breiman. 2001. Random Forests. *Machine Learning* 45, 1, 5–32. <https://doi.org/10.1023/A:1010933404324>
- [4] [4] S. Gamage and J. Samarabandu. 2020. Deep Learning Methods in Network Intrusion Detection: A Survey and an Objective Comparison. *Journal of Network and Computer Applications* 169, 102767.
- [5] [5] T. Saito and M. Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3, e0118432. <https://doi.org/10.1371/journal.pone.0118432>